

# 杭州电子科技大学

## 计算机组成原理（甲） 实 验 报 告

学 院 网络空间安全学院

专 业 网络空间安全

班 级 XX

学 号 XX

学生姓名

教师姓名

完成日期 2024.12.20

成 绩

## 实验十：R-I-J 型指令的 CPU

### 一、实验目的

- (1) 掌握 MIPS R 型、I 型和 J 型指令的综合数据通路设计
- (2) 掌握各种转移类指令的控制流和指令流的多路选通控制方法
- (3) 掌握 J 型、I 型和 R 型转移指令的指令格式和寻址方式，学习转移地址的产生方法
- (4) 掌握无条件转移指令和条件转移指令的实现方法
- (5) 编程实现 MIPS 的部分 J 型、I 型和 R 型转移指令的功能

### 二、实验原理

1. 实验九的基础上，预备实现 1 条 R 型转移指令、2 条 I 型条件转移指令和 2 条 J 型转移指令。
2. 与原理课相比，多了 3 条转移指令。

### 三、实验环境

所用电脑的软硬件配置：用 VMware 构建的 Windows 10 x64 的虚拟机，使用 iso 镜像文件 D:\Win10 镜像\Windows.iso

配置如下：

| 设备            | 摘要                    |
|---------------|-----------------------|
| 内存            | 4 GB                  |
| 处理器           | 2                     |
| 硬盘 (NVMe)     | 70 GB                 |
| CD/DVD (SATA) | 正在使用文件 D:\Win10镜像\... |
| 网络适配器         | NAT                   |
| USB 控制器       | 存在                    |
| 声卡            | 自动检测                  |
| 显示器           | 自动检测                  |

#### 内存

指定分配给此虚拟机的内存量。内存大小必须为 4 MB 的倍数。

此虚拟机的内存(M):  MB

128 GB -  
64 GB -  
32 GB -  
16 GB -  
8 GB -  
4 GB -  
2 GB -  
1 GB -  
512 MB -  
256 MB -  
128 MB -  
64 MB -  
32 MB -  
16 MB -  
8 MB -  
4 MB -

■ 最大建议内存  
(超出此大小可能发生内存交换。)

▲ 12.8 GB

■ 建议内存  
2 GB

■ 建议的最小客户机操作系统内存  
2 GB

实验所用的软件：ISE 14.7

#### 四、 主要操作步骤及实验结果记录（不能光截图，要有相应的文字说明）

（对实验过程中的主要操作步骤进行描述，并随时记录实验过程中观察到的结果，必要时可辅助截图）

##### 任务一：

实验八内容：设计实现一个单周期 CPU，实现 8 条 R 型指令：

- （1） 建立 R 型指令的数据通路；
- （2） 构造顶层模块，含部件：指令存储器（实验七）、PC 及自增电路（实验七），寄存器堆模块（实验四）、ALU 模块（实验三），指令译码与控制单元：新增，根据指令码和功能码，为数据通路上各部件发送控制信号（置位或复位）

功能模块说明：module R\_I\_J\_CPU(

```
    clk,rst,clk_m,
    Inst_code,PC,
    opcode,rs,rt,rd,shamt,func,imm,offset,
    ALU_F,ZF,OF,ALU_OP,
    imm_s,rt_imm_s,
    ALU_B,R_Data_A,W_Addr,W_Data,imm_kz,R_Data_B,M_R_Data,Write_Reg,Mem_Write,Mem_Addr,
    PC_s,PC_next,w_r_s,wr_data_s
);
input clk;//时钟
input rst;//清零
input clk_m;//Mem 时钟
output reg [31:0]PC;//当前 PC 地址
output reg [31:0]PC_next;//PC 转移地址
wire [31:0]PC_new; //PC+4
output [31:0]Inst_code;//取出的指令
output [5:0]opcode,func;//指令分段
output [4:0]rs,rt,rd,shamt;//指令分段
output [15:0]imm,offset;//指令分段
wire [25:0]address;//指令分段
output [31:0] ALU_F;//ALU 结果
output reg [2:0] ALU_OP;// ALU 运算的 OP
output ZF,OF;    //零标志位和溢出标志位
output reg Write_Reg;    //寄存器写入信号
output reg Mem_Write; //MEM 写入信号
output [31:0]R_Data_A; //REGS R_Data_A
output [31:0]R_Data_B; //REGS R_Data_B
output [31:0]M_R_Data; //MEM 读出数据 M_R_Data
output [7:0]Mem_Addr; //Mem 地址
output reg imm_s;      //控制立即数扩展方式的信号
output reg rt_imm_s; //控制 ALU_B 口输入数据的信号
output [4:0]W_Addr;    //REGS 写入地址
output [31:0]ALU_B;    //ALU_B 口数据
```

```

output [31:0]W_Data; //REGS 写入数据
output reg [31:0]imm_kz; //扩展后的立即数
output reg [1:0]PC_s; //控制 PC 下一次转移为何种方式的信号
output reg[1:0]w_r_s; //控制 w_addr 采用何种地址的信号
output reg[1:0]wr_data_s; //控制 W_Data 采用哪种数据的信号

```

8 条 R 型指令要求如下：

## ❖ （2） R 型指令集：

| 字段            | OP     | rs | rt | rd | shamt | func   | 功能描述                                    |
|---------------|--------|----|----|----|-------|--------|-----------------------------------------|
| 位数            | 6      | 5  | 5  | 5  | 5     | 6      |                                         |
| 汇编助记符         | 编码     |    |    |    |       |        |                                         |
| add rd,rs,rt  | 000000 | rs | rt | rd | 00000 | 100000 | 算术加: $rs + rt \rightarrow rd$           |
| sub rd,rs,rt  | 000000 | rs | rt | rd | 00000 | 100010 | 算术减: $rs - rt \rightarrow rd$           |
| and rd,rs,rt  | 000000 | rs | rt | rd | 00000 | 100100 | 位与: $rs \& rt \rightarrow rd$           |
| or rd,rs,rt   | 000000 | rs | rt | rd | 00000 | 100101 | 位或: $rs \mid rt \rightarrow rd$         |
| xor rd,rs,rt  | 000000 | rs | rt | rd | 00000 | 100110 | 位异或: $rs \oplus rt \rightarrow rd$      |
| nor rd,rs,rt  | 000000 | rs | rt | rd | 00000 | 100111 | 位或非: $\sim(rs \mid rt) \rightarrow rd$  |
| sltu rd,rs,rt | 000000 | rs | rt | rd | 00000 | 101011 | 无符号数小于则置位:<br>if(rs < rt)rd=1 else rd=0 |
| sllv rd,rt,rs | 000000 | rs | rt | rd | 00000 | 000100 | 逻辑左移:<br>(rt << rs) → rd                |

首先 Inst\_code 代表着当前取出的指令，R 型指令的具体内容主要由 func 来决定，根据这一点来对应输出相应的 ALU\_OP 值，

```

if (opcode==6'b000000) //R指令
begin
    case (func)
        6'b100000:begin ALU_OP=3'b100;end //add
        6'b100010:begin ALU_OP=3'b101;end //sub
        6'b100100:begin ALU_OP=3'b000;end //and
        6'b100101:begin ALU_OP=3'b001;end //or
        6'b100110:begin ALU_OP=3'b010;end //xor
        6'b100111:begin ALU_OP=3'b011;end //nor
        6'b101011:begin ALU_OP=3'b110;end //stlu
        6'b000100:begin ALU_OP=3'b111;end //sllv
    endcase
end

```

PC 及自增电路：设计时钟 clk,当 clk 上升沿时，启动指令存储器依据 PC 读指令，clk 高电平持续期间，完成 PC 值的自增，指令译码、寄存器读操作、随后完成 ALU 运算，clk 下降沿完成目的寄存器的写入，PC 值的更新和标志寄存器的更新。

ALU 模块：

```
module ALU(ALU_OP,A,B,F,ZF,OF);
    input  [2:0] ALU_OP;
    input  [31:0] A;
    input  [31:0] B;
    output [31:0] F;
    output  ZF;
    output  OF;
    reg [31:0] F;
    reg      C,ZF;

    always@(*)
    begin
        C=0;
        case(ALU_OP)
            3'b000:begin F=A&B; end
            3'b001:begin F=A|B; end
            3'b010:begin F=A^B; end
            3'b011:begin F=~(A|B); end
            3'b100:begin {C,F}=A+B; end
            3'b101:begin {C,F}=A-B; end
            3'b110:begin F=A<B; end
            3'b111:begin F=B<<A; end
        endcase
        ZF = F==0;
        end
        assign OF = ((ALU_OP==3'b100) || (ALU_OP==3'b101)) && (A[31] ^ B[31] ^ F[31] ^ C);
    endmodule
```

根据输入的 ALU\_OP 值来判断对 A 与 B 做什么操作，并将结果赋值给 F，具体说明见下表：

■ (1) 多功能ALU模块

ALU功能表

■ ALU通过3根控制线  
ALU\_OP[2:0]来选择其8种运算功能

| ALU_OP[2:0] | ALU | 操作说明                |
|-------------|-----|---------------------|
| 000         | and | 逻辑与操作               |
| 001         | or  | 逻辑或操作               |
| 010         | xor | 逻辑异或操作              |
| 011         | nor | 逻辑或非操作              |
| 100         | add | 算术加操作               |
| 101         | sub | 算术减操作               |
| 110         | slt | 若A<B，则输出1，<br>否则输出0 |
| 111         | sll | B逻辑左移A所指定的<br>位数    |

2020/9/20

4

ALU 位长为 3，共可以进行 8 种运算功能。  
寄存器堆模块：

```

module REGS(R_Data_A,R_Data_B,W_Data,R_Addr_A,R_Addr_B,W_Addr,Write_Reg,rst,clk);
input clk;//写入时钟信号
input rst;//清零信号
input Write_Reg;//写控制信号
input [4:0]R_Addr_A;//A端口读寄存器地址
input [4:0]R_Addr_B;//B端口读寄存器地址
input [4:0]W_Addr;//写寄存器地址
input [31:0]W_Data;//写入数据
output [31:0]R_Data_A;//A端口读出数据
output [31:0]R_Data_B;//B端口读出数据

integer i;
reg [31:0] REG_Files[0:31];
initial
    for(i=0;i<32;i=i+1) REG_Files[i]<=0;
always@(posedge clk or posedge rst)
begin
    if(rst)
        for(i=0;i<32;i=i+1) REG_Files[i]<=0;
    else
        if(Write_Reg&&W_Addr!=32'd0) REG_Files[W_Addr]<=W_Data;
end
assign R_Data_A=REG_Files[R_Addr_A];
assign R_Data_B=REG_Files[R_Addr_B];
endmodule

```

寄存器堆是一个 32 位长的 reg 类型数组，rst 信号用于初始化寄存器，clk 信号用于写入寄存器，当 Write\_Reg 为 0 时，进行读操作，当 Write\_Reg 为 1 时，进行写操作。

## 任务二：

实验九内容：在实验八的基础上，再行实现 MIPS 的 6 条 I 型指令，要求如下：

| 字段                | OP         | rs | rt | imm | 功能描述                                       |
|-------------------|------------|----|----|-----|--------------------------------------------|
| 位数                | 6          | 5  | 5  | 16  |                                            |
| 汇编助记符             | 编码         |    |    |     |                                            |
| addi rt, rs, imm  | 00100<br>0 | rs | rt | imm | 算术加：rs + imm→rt                            |
| andi rt, rs, imm  | 001100     | rs | rt | imm | 逻辑与：rs & imm→rt                            |
| xori rt, rs, imm  | 001110     | rs | rt | imm | 逻辑异或：rs⊕imm→rt                             |
| sltiu rt, rs, imm | 001011     | rs | rt | imm | 无符号数小于则置位：<br>if (rs < imm) rt=1 else rt=0 |

```

begin
case(opcode)
6'b001000:begin w_r_s=2'b01;imm_s=1;rt_imm_s=1;ALU_OP=3'b100;end //addi
6'b001100:begin w_r_s=2'b01;rt_imm_s=1;ALU_OP=3'b000; end //andi
6'b001110:begin w_r_s=2'b01;rt_imm_s=1;ALU_OP=3'b010;end //xori
6'b001011:begin w_r_s=2'b01;rt_imm_s=1;ALU_OP=3'b110; end //sltiu

```

这是 4 条立即数寻址的运算和传送指令，w\_r\_s;控制 w\_addr 采用何种地址的信号，这里均赋值为 01，也就是都选取了 rt 的地址，将运算结果写入 rt 中，对于第一条指令，imm\_s=1 意味着立即数要先进行符号扩展成 32 位后在进行运算，rt\_imm\_s 控制着 ALU\_B 口输入数据的信号，=1 说明将扩展好的立即数送入 ALU 的 B 端。

## ❖ 2.I型取数/存数指令及其数据通路

### •MIPS I型存储器访问指令格式及编码

| 字段                   | OP     | rs | rt | offset | 功能描述              |
|----------------------|--------|----|----|--------|-------------------|
| 位数                   | 6      | 5  | 5  | 16     |                   |
| 汇编助记符                | 编码     |    |    |        |                   |
| lw rt,<br>offset(rs) | 100011 | rs | rt | offset | 取数:(rs+offset)→rt |
| sw rt,<br>offset(rs) | 101011 | rs | rt | offset | 存数:rt→(rs+offset) |

```
6'b100011:begin w_r_s=2'b01;imm_s=1;rt_imm_s=1;wr_data_s=2'b01;ALU_OP=3'b100; end //lw
6'b101011:begin imm_s=1;rt_imm_s=1;ALU_OP=3'b100;Write_Reg=0;Mem_Write=1; end //sw
```

这是 2 条相对寄存器寻址的存数和取数指令，第一条中 w\_r\_s=1 决定将结果写入 rt 中，imm\_s=1 对立即数符号扩展成 32 位，并将扩展后的立即数输入 ALU 的 B 端，wr\_data\_s 控制 W\_Data 采用哪种数据的信号，这里赋值 01，采用数据存储器中的内容，完成取数；第二条中 Write\_Reg 控制着寄存器写入信号，Mem\_Write 控制着存储器写入信号，这里 Write\_Reg=0，Mem\_Write=1 是将数写入存储器中，完成存数。

#### 任务三：

实验九的基础上，预备实现 1 条 R 型转移指令、2 条 I 型条件转移指令和 2 条 J 型转移指令：

| R 型指令             | 字段 | OP     | rs      | rt   | rd     | shamt | func                                    | 功能描述              |
|-------------------|----|--------|---------|------|--------|-------|-----------------------------------------|-------------------|
|                   | 位数 | 6      | 5       | 5    | 5      | 5     | 6                                       |                   |
| jr rs             |    | 000000 | rs      | 5'b0 | 5'b0   | 5'b0  | 001000                                  | 无条件跳转:<br>rs → PC |
| I 型指令             | 字段 | OP     | rs      | rt   | offset |       | 功能描述                                    |                   |
|                   | 位数 | 6      | 5       | 5    | 16     |       |                                         |                   |
| beq rs, rt, label |    | 000100 | rs      | rt   | offset |       | 相等转移: if(rs=rt) PC+4+offset*4 → PC      |                   |
| bne rs, rt, label |    | 000101 | rs      | rt   | offset |       | 不相等转移: if(rs≠rt) PC+4+offset*4 → PC     |                   |
| J 型指令             | 字段 | OP     | address |      |        |       | 功能描述                                    |                   |
|                   | 位数 | 6      | 26      |      |        |       |                                         |                   |
| j label           |    | 000010 | address |      |        |       | 无条件跳转: {(PC+4) 高 4 位 ,address,0,0} → PC |                   |
| jal label         |    | 000011 | address |      |        |       | 无条件跳转并链接:<br>(PC+4) → \$31, 页面转移地址 → PC |                   |



```

6'b001000:begin Write_Reg=0;Mem_Write=0;PC_s = 2'b01; end    //jr
6'b000100:begin ALU_OP=3'b101;PC_s = (ZF)?2'b10:2'b00; Write_Reg = 1'b0;end    //beq
6'b000101:begin ALU_OP=3'b101;PC_s = (ZF)?2'b00:2'b10; Write_Reg = 1'b0;end    //bne
6'b000010:begin Write_Reg=0;Mem_Write=0;PC_s = 2'b11; end    //j
6'b000011:begin w_r_s=2'b10;wr_data_s=2'b10;Write_Reg=1;Mem_Write=0;PC_s = 2'b11; end    //jal

```

这里 PC\_s 控制 PC 下一次转移为何种方式的信号

```

    case (PC_s)    //根据PC_s , 赋值PC_next
2'b00:    PC_next = PC_new;
2'b01:    PC_next = R_Data_A;
2'b10:    PC_next = PC_new + (imm_kz<<2);
2'b11:    PC_next = {PC_new[31:28],address,2'b00};
    endcase

```

在 jr 指令中，既不写入寄存器也不写入存储器，直接由寄存器中的 R\_Data\_A 给出 PC 的跳转地址；beq 为相等转移指令，bne 为不相等转移指令，ALU\_OP 表示 rs 与 rt 做减法，然后用 ZF 零标志位去判断，当 rs=rt 时，发生相等转移，将更新后的 PC 值与左移后的立即数相加得到新 PC 的跳转地址，rs≠rt 时同理；j 和 jal 均为无条件必转指令，相比 j 指令，jal 指令不仅转移，且在转移前，将 jal 指令的下一条指令的地址保存到编号为 31 的 \$ra(\$31) 寄存器。

#### 任务四：指令测试：

测试部分代码：



```

// Instantiate the Unit Under Test (UUT)
R_I_J_CPU uut (
    .clk(clk),
    .rst(rst),
    .clk_m(clk_m),
    .Inst_code(Inst_code),
    .PC(PC),
    .opcode(opcode),
    .rs(rs),
    .rt(rt),
    .rd(rd),
    .shamt(shamt),
    .func(func),
    .imm(imm),
    .offset(offset),
    .ALU_F(ALU_F),
    .ZF(ZF),
    .OF(OF),
    .ALU_OP(ALU_OP),
    .imm_s(imm_s),
    .rt_imm_s(rt_imm_s),
    .ALU_B(ALU_B),
    .R_Data_A(R_Data_A),
    .W_Addr(W_Addr),
    .W_Data(W_Data),
    .imm_kz(imm_kz),
    .R_Data_B(R_Data_B),
    .M_R_Data(M_R_Data),
    .Write_Reg(Write_Reg),
    .Mem_Write(Mem_Write),
    .Mem_Addr(Mem_Addr),
    .PC_s(PC_s),
    .PC_next(PC_next),
    .w_r_s(w_r_s),
    .wr_data_s(wr_data_s)
);

    always #13 clk_m=~clk_m;
always #47 clk=~clk;

initial begin

    clk = 0;
    rst = 1;
    clk_m = 0;
    #3;
    rst=0;
end

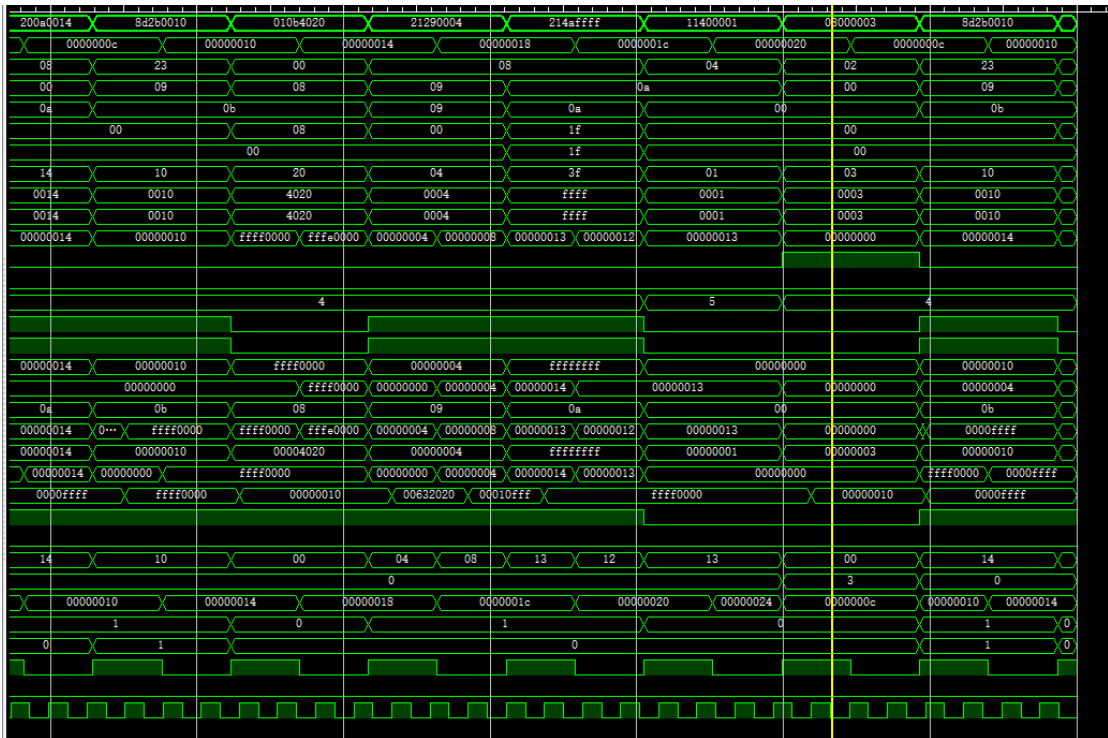
endmodule

```

测试程序 1:

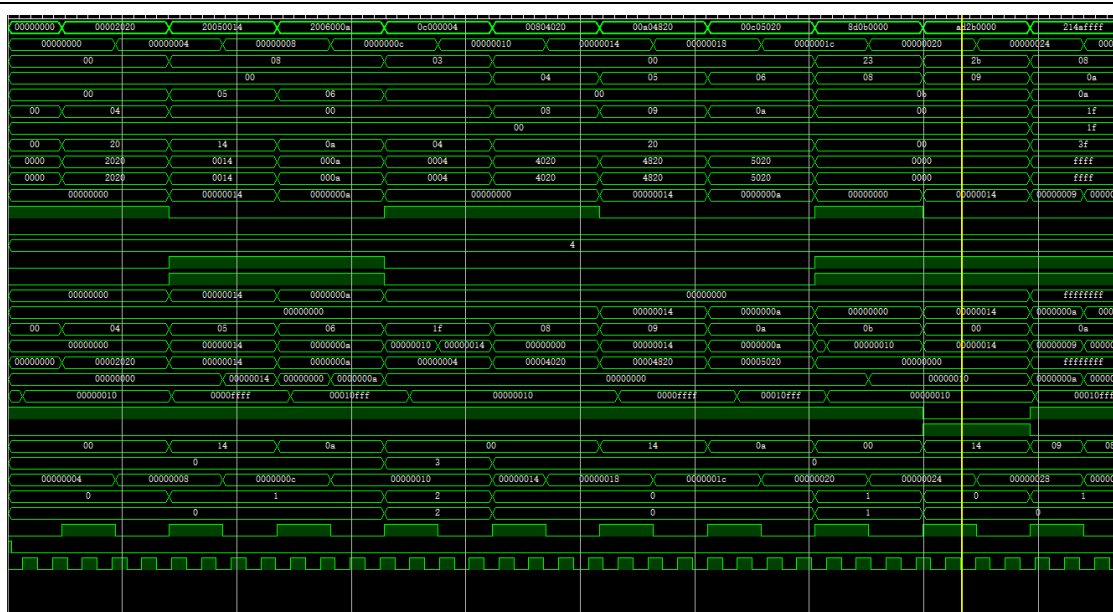
| 序号 | 指令       | ALU_F    | 标志 |    | 指令类型及功能    |
|----|----------|----------|----|----|------------|
|    |          |          | OF | ZF |            |
| 1  | 00004020 | 00000000 | 0  | 1  | R 型指令, add |

|    |          |          |   |   |                |
|----|----------|----------|---|---|----------------|
| 2  | 00004820 | 00000000 | 0 | 1 | R 型指令, add     |
| 3  | 200a0014 | 00000014 | 0 | 0 | I 型指令 addi 算数加 |
| 4  | 8d2b0010 | 00000010 | 0 | 0 | I 型指令 lw, 取数   |
| 5  | 010b4020 | ffff0000 | 0 | 0 | R 型指令 add      |
| 6  | 21290004 | 00000004 | 0 | 0 | I 型指令 addi 算数加 |
| 7  | 214affff | 00000013 | 0 | 0 | I 型指令 addi 算数加 |
| 8  | 11400001 | 00000013 | 0 | 0 | I 型指令 beq 相等转移 |
| 9  | 08000003 | 00000000 | 0 | 0 | J 型指令 j 无条件跳转  |
| 10 | 8d2b0010 | 00000014 | 0 | 0 | 由上条指令跳转        |



测试程序 2:

| 序号 | 指令       | ALU_F    | 标志 |    | 指令类型及功能                |
|----|----------|----------|----|----|------------------------|
|    |          |          | OF | ZF |                        |
| 1  | 00002020 | 00000000 | 0  | 1  | R 型指令, add             |
| 2  | 20050014 | 00000014 | 0  | 0  | I 型指令 addi 算数加         |
| 3  | 2006000a | 0000000a | 0  | 0  | I 型指令 addi 算数加         |
| 4  | 0c000004 | 00000010 | 0  | 1  | J 型指令 jal 无条件跳转<br>并链接 |
| 5  | 00804020 | 00000000 | 0  | 1  | R 型指令, add             |
| 6  | 00a04820 | 00000014 | 0  | 0  | R 型指令, add             |
| 7  | 00c05020 | 0000000a | 0  | 0  | R 型指令, add             |
| 8  | 8d0b0000 | 00000013 | 0  | 1  | I 型指令 lw, 取数           |
| 9  | ad2b0000 | 00000000 | 0  | 0  | I 型指令 sw, 存数           |
| 10 | 214affff | 00000009 | 0  | 0  | I 型指令 addi 算数加         |
| 11 | 1540fffc | 00000009 | 0  | 0  | I 型指令 bne 不相等转移        |
| 12 | 8d0b0000 | 00000000 | 0  | 1  | 由上条指令跳转                |



#### 任务五：思考与探索

（2）转移指令的 **offset** 字段和 **address** 字段的编码，计算出转移地址，观察是否和你的转移目标地址一致

执行 1540fffc 这条不相等转移指令时， $PC_{s}=2$ ，执行  $PC_{next} = PC_{new} + (imm_{kz} \ll 2)$ ；此时 PC 为 00000028，则  $PC_{new}=0000002c$ ，imm 为 fffc，经过 0 扩展为 32 位后左移两位，最终根据等式计算可得  $PC_{next}=0004001c$ ，与转移目标地址一致。

（3）I 型指令 **bltzal rs, label**（branch if less than zero, and link）的功能是：

- 若寄存器 **rs** 小于 0，则转移并链接，相对当前指令（**PC+4**）转移的指令数由 **offset** 来决定。它的 **OP** 编码为 **6'b000001**，**rt** 字段为 **5'b10000**
- 试着实现该指令，谈谈你的实现方法。

要判断 **rs** 的正负，通过译码器模块判断 **R\_DATA\_A** 的数据的正负，如果是负数，此条指令的实现与 **jal** 相同。

## 五、 实验分析总结及心得

（结合所学知识对实验过程中观察到的实验结果进行分析总结，以便加深对知识的理解，并总结通过实验学到的知识或技术）

在本次实验中，我们通过使用 Verilog 语言编写了 R 型、I 型和 J 型转移指令的功能，并进行了仿真验证。实验的主要目标是加深对 MIPS 指令集体系结构中转移指令的理解，特别是无条件转移和条件转移指令的实现。实验过程中，遇到了一些挑战。首先，对于不同类型的指令（R 型、I 型、J 型），如何正确地设计数据通路和控制流的多路选通控制是最大的难点之一。通过仔细分析每种指令的格式和寻址方式，逐步理清了它们的控制信号如何产生，尤其是在条件转移指令中的判断条件如何影响控制信号。

其次，如何有效地在 Verilog 中实现指令的功能也给我带来了一些困惑。在设计转移指令时，尤其是 J 型和 I 型转移指令的地址计算部分，我们花了较多时间调试地址计算的部分，确保程序跳转的准确性。通过对每个模块进行单元测试，逐步定位并修复了指令执行中的错误。

通过这个实验，我们不仅掌握了 MIPS 指令集的各种转移指令，还进一步提高了 Verilog 编程的能力，尤其是多路选择器和控制信号生成的设计技巧。最终的仿真结果证明了我们的设计是成功的，这让我们对硬件设计和指令集体系有了更深刻的理解和认识。

## 六、 分工